

El Asistente con IA

La vista asistente() explicada línea por línea

Proyecto Biblioteca Comunitaria
Trabajo práctico de Desarrollo Web

Componente	Archivo	Rol
Ruta	sitio/libros/urls.py	Asocia /asistente/ con la vista
Vista	sitio/libros/views.py	Recibe la pregunta, llama al servicio, arma el context
Servicio	sitio/libros/servicios.py	Arma el contexto, llama a la API de IA, maneja errores
Configuración	sitio/biblioteca/settings.py	Lee la clave, la URL, el modelo y el timeout
Template	sitio/libros/templates/libros/asistente.html	Muestra el formulario y la respuesta

1. Qué es esta vista y cómo encaja

La vista `asistente()` es la sexta vista de la app `libros`. Su trabajo es simple de enunciar y complejo de lograr: **dejar que un usuario le pregunte en lenguaje natural a una inteligencia artificial cosas sobre el catálogo** (qué libros hay, quién tiene uno prestado, cómo se devuelve un préstamo) y mostrar la respuesta.

La particularidad es que la IA es un sistema **externo**, como el microservicio de reseñas. La vista no sabe cómo se genera la respuesta: solo sabe pedirla y mostrarla. Todo el conocimiento de la IA —el formato del pedido, la URL, la clave— vive en `servicios.py`, y la configuración en `settings.py`.

El requisito que cumple

La consigna pedía que el proyecto consumiera una **API de inteligencia artificial** para responder preguntas sobre el catálogo. El asistente usa Gemini, de Google, pero el código no conoce a Gemini: conoce una URL, un modelo y una clave.

La idea central

La IA se trata como un recurso remoto y falible. Si la clave falta o la API se cae, la página se muestra igual con un aviso; nunca un error 500. Es el mismo criterio de degradación controlada que ya usan las reseñas.

2. La ruta: cómo se llega a la vista

sitio/libros/urls.py (líneas 7-13)

```
from django.urls import path
from . import views

app_name = "libros"

urlpatterns = [
    path("", views.inicio, name="inicio"),
    path("libro/<int:libro_id>/", views.detalle_libro, name="detalle_libro"),
    path("categoria/<int:categoria_id>/", views.libros_por_categoria, name="por_categoria"),
    path("autor/<int:autor_id>/", views.libros_por_autor, name="por_autor"),
    path("libro/<int:libro_id>/resenas/", views.resenas_libro, name="resenas_libro"),
    path("asistente/", views.asistente, name="asistente"),
]
```

La línea que importa es la última: `path("asistente/", views.asistente, name="asistente")`. Tres decisiones que vale la pena leer:

1. La URL `/asistente/` no lleva parámetro: la vista no necesita un id en la ruta porque la pregunta viaja por el **cuerpo del POST**, no por la URL. A diferencia de `libro/<int:libro_id>/`, acá no hay nada dinámico que capturar.
2. La misma URL atiende **dos métodos**: GET muestra el formulario vacío y POST procesa la pregunta. La vista decide según `request.method`. Esto no requiere nada especial en `urls.py`: el enrutador de Django no distingue métodos.
3. El `name="asistente"` es lo que permite escribir `{% url 'libros:asistente' %}` en los templates sin hardcodear la URL. El prefijo `libros:` viene de `app_name = "libros"`. Ese link está en la barra de `base.html`.

3. La vista: views.py

La vista entera, tal como está en el archivo, son 20 líneas. La leemos de arriba a abajo.

[sitio/libros/views.py](#) (líneas 104-123)

```
def asistente(request):
    """Muestra el formulario (GET) y consulta la IA con el contexto del proyecto (POST)."""
    respuesta = ""
    error = ""

    if request.method == "POST":
        pregunta = request.POST.get("pregunta", "").strip()
        if not pregunta:
            error = "Escribí una pregunta primero"
        else:
            try:
                respuesta = servicios.preguntar_al_asistente(pregunta)
            except servicios.IANoDisponible as fallo:
                error = f"No se pudo consultar a la IA ({fallo})"

    context = {
        "respuesta": respuesta,
        "error": error,
    }
    return render(request, "libros/asistente.html", context)
```

3.1 Los dos acumuladores

La vista arranca inicializando dos variables en vacío:

```
respuesta = ""
error = ""
```

Son **acumuladores**: se llenan a lo largo del flujo y se mandan siempre al template. Gracias a estos valores por defecto, el template puede asumir que `respuesta` y `error` siempre existen en el context. En un GET —donde todavía no hay nada que mostrar— quedan como cadenas vacías, y el template las ignora con sus `{% if %}`.

3.2 GET vs. POST

El `if request.method == "POST"` separa los dos usos de la misma URL. En **GET** (abrir la página) el bloque se salta entero y la vista va directo a renderizar el formulario. En **POST** (enviar el formulario) se ejecuta la lógica de consulta.

Por qué no usar el GET para preguntar

Si la pregunta fuera por GET, quedaría visible en la URL, en el historial y en los logs del servidor. Además, un GET debe ser una operación que no modifica nada. Acá cada consulta dispara una llamada a una API de pago, así que debe ser un POST.

3.3 Leer y limpiar la pregunta

```
pregunta = request.POST.get("pregunta", "").strip()
```

`request.POST` es un diccionario con los campos del formulario enviado. `.get("pregunta", "")` lee el campo llamado `pregunta` (el `name` del `textarea`) y devuelve una cadena vacía si faltara. El `.strip()` quita espacios al inicio y al final, de modo que " " se convierte en "".

3.4 Validar y consultar

```
if not pregunta:
    error = "Escribí una pregunta primero"
else:
    try:
        respuesta = servicios.preguntar_al_asistente(pregunta)
    except servicios.IANoDisponible as fallo:
        error = f"No se pudo consultar a la IA ({fallo})"
```

Hay dos caminos de fallo y uno de éxito:

1. Si la pregunta quedó vacía tras el `strip`, no vale la pena gastar una llamada a la API: se setea el error y se termina.
2. Si hay pregunta, se llama a `servicios.preguntar_al_asistente(pregunta)`. Esa función puede lanzar `IANoDisponible`, la excepción propia del proyecto para cuando la IA falla. Si ocurre, la vista la atrapa y convierte el fallo en un mensaje legible.
3. Si todo sale bien, `respuesta` queda con el texto de la IA. Nótese el `f-string` del error: incluye el detalle de la excepción entre paréntesis, lo que ayuda a diagnosticar sin mostrar un `traceback`.

3.5 El context y el render

```
context = {
    "respuesta": respuesta,
    "error": error,
}
return render(request, "libros/asistente.html", context)
```

Cumple el patrón de tres pasos que exige el proyecto: el `context` es una variable con nombre y se usa el shortcut `render()`. El template recibe exactamente dos variables: lo que respondió la IA (o vacío) y el error (o vacío). No hay más estado que ese.

4. El servicio: servicios.py

La vista es delgada porque toda la complejidad externa está en `servicios.py`. Este módulo concentra **todo** lo que el proyecto sabe sobre la API de IA, igual que ya hacía con el microservicio de reseñas. Hay tres piezas: la excepción propia, el cliente HTTP genérico `_pedir()`, y las dos funciones específicas del asistente: `armar_contexto_biblioteca()` y `preguntar_al_asistente()`.

4.1 La excepción propia

`sitio/libros/servicios.py` (líneas 21-22)

```
class IANoDisponible(Exception):
    """La API de IA no respondió, tardó demasiado o falta la clave en biblioteca/.env."""
```

Es una excepción propia, del mismo estilo que `MicroservicioNoDisponible`. Define un vocabulario del problema: *"la IA no está"*. Todas las fallas técnicas del lado de la IA —red caída, timeout, respuesta malformada, falta de clave— se traducen a esta única excepción. Así la vista atrapa **una** cosa, sin saber nada de `urllib` ni de HTTP.

4.2 El cliente HTTP genérico

`sitio/libros/servicios.py` (líneas 25-40)

```
def _pedir(url, datos=None, timeout=5, error_cls=MicroservicioNoDisponible):
    """Hace la petición HTTP y devuelve el JSON ya parseado."""
    cuerpo = None
    cabeceras = {"Accept": "application/json"}

    if datos is not None:
        cuerpo = json.dumps(datos).encode("utf-8")
        cabeceras["Content-Type"] = "application/json"

    peticion = urllib.request.Request(url, data=cuerpo, headers=cabeceras)

    try:
        with urllib.request.urlopen(peticion, timeout=timeout) as respuesta:
            return json.loads(respuesta.read().decode("utf-8"))
    except (urllib.error.URLError, TimeoutError, ValueError) as error:
        raise error_cls(str(error)) from error
```

Esta función es la misma que usa el microservicio de reseñas, con dos diferencias de diseño para servir a la IA:

1. El parámetro `timeout` ahora es configurable: el asistente le pasa 15 segundos, porque una consulta de IA tarda más que una lectura de reseñas.
2. El parámetro `error_cls` dice qué excepción lanzar. El asistente pasa `IANoDisponible` para que el error llegue con el nombre correcto.

El cuerpo: si `datos` no es `None`, se serializa a JSON con `json.dumps` y se manda en el cuerpo del request con `Content-Type: application/json`. Eso es lo que convierte a la petición en un POST. El `with` garantiza cerrar la conexión, y los tres errores atrapados (red, timeout, JSON inválido) se convierten en `error_cls`.

4.3 Armar el contexto del catálogo

Una IA externa no sabe nada del proyecto. Para que responda con datos reales y no invente, la función `armar_contexto_biblioteca()` arma un **texto plano** con la descripción de la app, cómo funciona un préstamo y los datos actuales de la base. Ese texto viaja con la pregunta.

`sitio/libros/servicios.py` (líneas 61-124, resumido)

```
def armar_contexto_biblioteca():
    """Devuelve un texto con el funcionamiento de la app y los datos actuales para la IA."""
    lineas = []

    lineas.append("DESCRIPCION DE LA APP:")
    lineas.append(
        "La biblioteca comunitaria es un sistema web en Django. Un lector "
        "registrado se lleva un libro prestado: eso se llama un préstamo. "
        "Cada libro tiene un campo 'disponible': True significa que está en la "
        "biblioteca y se puede prestar; False significa que alguien ya lo tiene. ..."
    )

    lineas.append("COMO FUNCIONA UN PRESTAMO:")
    lineas.append(
        "- Prestar: se elige un lector y un libro disponible, se crea el "
        "préstamo con estado 'activo' y el libro pasa a disponible=False. ..."
    )
    lineas.append(
        "- Devolver: el préstamo activo pasa a estado 'devuelto', se guarda la "
        "fecha de devolución y el libro vuelve a disponible=True. ..."
    )

    lineas.append("QUE SE PUEDE HACER EN LA APP:")
    lineas.append("- Ver el catálogo completo y si cada libro está disponible o prestado.")
    lineas.append("- Filtrar libros por categoría y por autor.")
    lineas.append("- Ver el detalle de cada libro, sus autores y sus reseñas.")
    lineas.append("- Registrar y devolver préstamos, y verlos por estado.")
    lineas.append("- Ver el historial de préstamos de cada lector.")
    return "
".join(lineas)
```

Después de la descripción estática, la función **consulta el ORM** y vuelca los datos reales:

`sitio/libros/servicios.py` (líneas 99-124)

```

lineas.append("DATOS ACTUALES DEL PROYECTO:")
lineas.append("LIBROS:")
for libro in Libro.objects.all():
    autores = ", ".join(str(autor) for autor in libro.autores.all())
    estado = "disponible" if libro.disponible else "prestado"
    lineas.append(
        f"- {libro.titulo} ({libro.anio_publicacion}), "
        f"categoria {libro.categoria.nombre}, autores: {autores}, {estado}"
    )

lineas.append("CATEGORIAS:")
for categoria in Categoria.objects.all():
    lineas.append(f"- {categoria.nombre}")

lineas.append("LECTORES:")
for lector in Lector.objects.all():
    lineas.append(f"- {lector.nombre} ({lector.email})")

lineas.append("PRESTAMOS:")
for prestamo in Prestamo.objects.all():
    lineas.append(
        f"- {prestamo.libro.titulo} -> {prestamo.lector.nombre} "
        f"({prestamo.estado}, desde {prestamo.fecha_prestamo})"
    )

return "
".join(lineas)

```

La función importa los cuatro modelos del proyecto: Libro y Categoria de libros, y Lector y Prestamo de prestamos. Es el único momento en que el asistente toca la base local: lee, no escribe.

Por qué el contexto se arma con datos reales

Si solo se le diera a la IA la descripción de la app, inventaría datos. El contexto con los libros y préstamos actuales le da **anclaje**: la consigna del mensaje de sistema le prohíbe responder con información que no esté en el contexto. Así el asistente habla de los 5 libros que hay de verdad, no de los que imagina.

4.4 La función que llama a la IA

sitio/libros/servicios.py (líneas 127-161)

```
def preguntar_al_asistente(pregunta):
    """Envía la pregunta con el contexto del catálogo a la API de IA y devuelve la respuesta."""
    if not settings.IA_API_KEY:
        raise IANoDisponible("falta IA_API_KEY en biblioteca/.env")

    contexto = armar_contexto_biblioteca()
    mensaje_sistema = (
        "Sos el asistente de la Biblioteca Comunitaria, un sistema web de "
        "préstamo de libros. Respondé SOLO usando la información que te doy en "
        "el contexto (el funcionamiento de la app y sus datos actuales), nunca "
        "inventes datos ni funciones que no estén en el contexto. ... "
        "Respondé en español, breve y en el mismo idioma que la pregunta."
    )

    url = f"{settings.IA_API_URL}/models/{settings.IA_MODEL}:generateContent?key={settings.IA_API_KEY}"
    datos = {
        "contents": [
            {
                "parts": [
                    {"text": mensaje_sistema},
                    {"text": f"CONTEXTO DEL PROYECTO:\n{contexto}\n\nPREGUNTA: {pregunta}"},
                ]
            },
        ],
        "generationConfig": {"temperature": 0.2},
    }

    respuesta = _pedir(url, datos, timeout=settings.IA_TIMEOUT, error_cls=IANoDisponible)
    return respuesta["candidates"][0]["content"]["parts"][0]["text"].strip()
```

4.4.1 La clave como guarda

La primera línea es una **validación de configuración**:

```
if not settings.IA_API_KEY:
    raise IANoDisponible("falta IA_API_KEY en biblioteca/.env")
```

Si no hay clave configurada, no tiene sentido armar la URL (la petición fallaría de todas formas). Mejor fallar temprano, con un mensaje que dice exactamente qué falta. La vista atrapa esto como un `IANoDisponible` más y muestra el aviso.

4.4.2 El mensaje de sistema

El `mensaje_sistema` es la **instrucción de rol** para la IA: le dice quién es, qué debe responder y qué debe evitar. Es la frontera técnica del problema de las alucinaciones: prohibir explícitamente inventar datos y obligarla a basarse solo en el contexto. También le pide responder en español.

4.4.3 El formato del pedido a Gemini

```
url = f"{settings.IA_API_URL}/models/{settings.IA_MODEL}:generateContent?key={settings.IA_API_KEY}"
datos = {
    "contents": [
        {
            "parts": [
                {"text": mensaje_sistema},
                {"text": f"CONTEXTO DEL PROYECTO:\n{contexto}\n\nPREGUNTA: {pregunta}"},
            ]
        }
    ],
    "generationConfig": {"temperature": 0.2},
}
```

Cada proveedor de IA define su propio protocolo. Gemini usa `:generateContent` como acción, pasa la clave por la URL con `?key=...` (no por una cabecera de autorización, como haría OpenAI) y espera los textos dentro de `contents` → `parts` → `text`.

La pregunta y el contexto viajan **en el mismo part**, separados por etiquetas legibles: `CONTEXTO DEL PROYECTO:` y `PREGUNTA:`. El mensaje de sistema va en un `part` anterior. `temperature: 0.2` pide respuestas predecibles y apegadas a los datos: no queremos que el asistente sea creativo, queremos que sea exacto.

4.4.4 Leer la respuesta

```
respuesta = _pedir(url, datos, timeout=settings.IA_TIMEOUT, error_cls=IANoDisponible)
return respuesta["candidates"][0]["content"]["parts"][0]["text"].strip()
```

Gemini devuelve un JSON anidado: una lista de `candidates` (respuestas posibles, tomamos la primera), cada una con su `content`, que tiene `parts` (fragmentos de texto) y en el primero su `text`. El `.strip()` final limpia espacios. Nótese cómo se navega el JSON: tres índices anidados con sus claves literales, que es la parte más frágil del código: si Gemini cambia el formato, esto rompe con un `KeyError` que `_pedir` traduciría a `IANoDisponible`.

5. La configuración: settings.py

El asistente lee cuatro valores de configuración. La regla del proyecto es que las credenciales **nunca** se escriben en el código: se leen de variables de entorno, y en local desde un archivo `.env` que `settings.py` parsea solo.

`sitio/biblioteca/settings.py` (líneas 224-229)

```
IA_API_KEY = os.environ.get('IA_API_KEY') or _ENV_LOCAL.get('IA_API_KEY', '')
IA_API_URL = os.environ.get('IA_API_URL') or _ENV_LOCAL.get(
    'IA_API_URL', 'https://generativelanguage.googleapis.com/v1beta',
)
IA_MODEL = os.environ.get('IA_MODEL') or _ENV_LOCAL.get('IA_MODEL', 'gemini-2.5-flash')
IA_TIMEOUT = 15 # segundos: una consulta de IA tarda más que el microservicio
```

El patrón `os.environ.get(...)` or `_ENV_LOCAL.get(...)` es la jerarquía de precedencia: si la variable ya está en el entorno (por ejemplo en Vercel), gana esa; si no, se busca en el `.env` local; y solo si no hay nada, se usa el valor por defecto. Así la misma configuración funciona en la nube y en la máquina del desarrollador sin cambiar una línea.

5.1 De dónde sale `_ENV_LOCAL`

`sitio/biblioteca/settings.py` (líneas 207-222)

```
def _leer_env_local():
    """Lee las variables del .env de esta carpeta y las devuelve en un dict."""
    ruta = Path(__file__).resolve().parent / '.env'
    variables = {}
    if not ruta.exists():
        return variables
    for linea in ruta.read_text(encoding='utf-8').splitlines():
        linea = linea.strip()
        if not linea or linea.startswith('#') or '=' not in linea:
            continue
        nombre, _, valor = linea.partition('=')
        variables[nombre.strip()] = valor.strip().strip('"\'')
    return variables

_ENV_LOCAL = _leer_env_local()
```

Lee el archivo `biblioteca/.env` —que no se versiona— línea por línea, ignora comentarios y líneas sin `=`, y guarda pares clave/valor. El `.strip('"\'')` quita las comillas con las que se envuelven los valores en el archivo, porque la shell las necesitaría pero el valor real no las incluye.

6. El template: asistente.html

`sitio/libros/templates/libros/asistente.html`

```
{% extends "base.html" %}

{% block title %}Asistente con IA{% endblock %}

{% block content %}
    <h2>Asistente con IA</h2>
    <p>Preguntá por los libros, autores, categorías, lectores o préstamos de la biblioteca.</p>
    <form method="post">
        {% csrf_token %}
        <p><label for="pregunta">Tu pregunta:</label></p>
        <p><textarea id="pregunta" name="pregunta" rows="4" cols="60" required></textarea></p>
        <p><button type="submit">Preguntar</button></p>
    </form>

    {% if error %}
        <p><strong>{{ error }}</strong></p>
    {% endif %}

    {% if respuesta %}
        <hr>
        <h3>Respuesta</h3>
        <p>{{ respuesta|linebreaks }}</p>
    {% endif %}
{% endblock %}
```

El template no consulta la base ni llama a la IA: solo muestra las dos variables del context. Cada bloque tiene su lógica:

1. `{% extends "base.html" %}` hereda la estructura común (barra de navegación, título, pie). El bloque `{% block content %}` es donde este template mete su contenido.
2. El formulario usa `method="post"`, necesario para que la vista entre en la rama POST. El `{% csrf_token %}` es la protección CSRF de Django: sin él, el POST sería rechazado con un 403. El `textarea` tiene `name="pregunta"`, que es la clave que la vista lee con `request.POST.get("pregunta")`, y `required` para que el navegador no deje enviar vacío.
3. El error se muestra solo si `{% if error %}` tiene contenido. La respuesta, solo si `{% if respuesta %}`. El filtro `|linebreaks` convierte los saltos de línea del texto en `<p>` o `<br>`, porque la IA suele responder en párrafos y el HTML plano los ignoraría.

Ambos `{% if %}` son posibles gracias a los acumuladores de la vista: `error` y `respuesta` siempre existen en el context, aunque estén vacíos.

7. Cómo se prueba

El asistente consulta una API externa, así que el test **no puede** llamar a Gemini de verdad (sería lento, costoso y no determinista). La técnica es el mock: se reemplaza la función del servicio por un falso y se prueba cómo reacciona la vista. Hay tres tests en `sitio/libros/tests.py`.

7.1 Cuando la IA responde

```
def test_muestra_la_respuesta_que_devuelve_la_ia(self):
    with patch(
        "libros.servicios.preguntar_al_asistente",
        return_value="Hay 3 libros de Ficción.",
    ):
        respuesta = self.client.post(
            "/asistente/",
            {"pregunta": "¿Cuántos libros de ficción hay?"},
        )

    self.assertEqual(respuesta.status_code, 200)
    self.assertContains(respuesta, "Hay 3 libros de Ficción.")
```

`patch` reemplaza `preguntar_al_asistente` con un falso que devuelve un texto fijo. La vista se comporta igual, pero no hay red. El test hace un POST real al cliente de pruebas de Django y verifica que el HTML contiene la respuesta.

7.2 Cuando la IA falla

```
def test_si_la_ia_no_responde_la_pagina_igual_carga(self):
    fallo = servicios.IANoDisponible("timeout")

    with patch("libros.servicios.preguntar_al_asistente", side_effect=fallo):
        respuesta = self.client.post(
            "/asistente/",
            {"pregunta": "¿Qué libros hay?"},
        )

    self.assertEqual(respuesta.status_code, 200)
    self.assertContains(respuesta, "No se pudo consultar a la IA")
```

Este test protege la degradación controlada: si la IA falla, la página sigue devolviendo 200 y muestra el aviso. Es la misma filosofía que el microservicio de reseñas: lo remoto falla y la página no debe romperse.

7.3 El contexto del catálogo

```
def test_armar_contexto_biblioteca_incluye_los_datos_del_orm(self):
    categoria = Categoria.objects.create(nombre="Ficción")
    libro = Libro.objects.create(
        titulo="Ficciones", isbn="9788420633997", anio_publicacion=1944,
        paginas=176, categoria=categoria,
    )

    contexto = servicios.armar_contexto_biblioteca()

    self.assertIn("LIBROS:", contexto)
    self.assertIn("Ficciones", contexto)
    self.assertIn("categoria Ficción", contexto)
```

Este test sí crea datos reales en la base de prueba y verifica que `armar_contexto_biblioteca()` los vuelca al texto. Garantiza que la IA siempre reciba el catálogo real.

8. El recorrido completo de una pregunta

Todo junto, esto es lo que pasa cuando alguien escribe una pregunta y la envía:

#	Qué ocurre
1	El usuario abre /asistente/: GET, formulario vacío.
2	Escribe una pregunta y pulsa «Preguntar»: el navegador manda un POST con el campo pregunta y el token CSRF.
3	Django valida el CSRF y enruta al view asistente(request).
4	La vista lee y limpia la pregunta: request.POST.get("pregunta").strip().
5	Si quedó vacía, setea el error y salta al render.
6	Si hay pregunta, llama a servicios.preguntar_al_asistente(pregunta).
7	La función verifica la clave, arma el contexto del catálogo con armar_contexto_biblioteca() (que consulta los 4 modelos), y compone el mensaje de sistema.
8	Arma la URL de Gemini y el payload JSON, y llama a _pedir() con 15 segundos de timeout.
9	_pedir() hace el POST por urllib y parsea el JSON de la respuesta.
10	Se extrae el texto del primer candidate y se devuelve como string.
11	La vista guarda la respuesta en el acumulador y arma el context.
12	El template renderiza la respuesta con linebreaks y el usuario la lee en pantalla.

Qué pasa si algo falla

Falla	Dónde se detecta	Qué ve el usuario
Falta la clave	preguntar_al_asistente (línea 129)	«No se pudo consultar a la IA (falta IA_API_KEY...)»
La API no responde o tarda +15s	_pedir (timeout)	«No se pudo consultar a la IA (...timeout...)»
La pregunta está vacía	la vista (if not pregunta)	«Escribí una pregunta primero»
Respuesta con formato inesperado	_pedir (ValueError)	«No se pudo consultar a la IA (...bad JSON...)»

9. Lo que conviene llevarse

1. La vista es un mediador delgado

No conoce a Gemini, ni la clave, ni el formato del JSON. Recibe una pregunta, llama a una función del servicio y muestra el resultado. Toda la complejidad externa vive en `servicios.py`.

2. La IA se trata como un recurso falible

Timeout, excepción propia y degradación controlada. Sin clave o sin API, la página carga igual con un aviso. Nunca un 500.

3. El contexto ancla a la realidad

El texto que se le manda a la IA se arma con datos reales del ORM. El mensaje de sistema le prohíbe inventar. Eso convierte a un modelo generalista en un asistente del catálogo.

4. La configuración no viaja en el código

Clave, URL, modelo y timeout se leen por variable de entorno con fallback a un `.env` no versionado y a valores de desarrollo. En Vercel se cargan desde el panel.

Archivos de referencia: `sitio/libros/views.py`, `sitio/libros/servicios.py`, `sitio/biblioteca/settings.py`, `sitio/libros/urls.py`, `sitio/libros/templates/libros/asistente.html`, `sitio/libros/tests.py`

10. Glosario de términos

Los términos que aparecen en este documento, explicados en el contexto del asistente.

La vista y el flujo

Vista	Función de Python que recibe una petición HTTP y devuelve una respuesta. <code>asistente()</code> sigue el patrón de tres pasos del proyecto: consumir datos, armar el context, renderizar.
GET / POST	Los dos métodos HTTP. GET pide una página sin efectos; POST envía datos para procesarlos. La misma URL <code>/asistente/</code> responde a ambos, y la vista distingue por <code>request.method</code> .
Context	Diccionario que la vista le pasa al template. Acá lleva exactamente dos claves: <code>respuesta</code> y <code>error</code> .
Template	HTML con plantillas de Django. Solo muestra las variables del context; no consulta la base ni llama a servicios.
CSRF token	Token de seguridad que Django exige en los POST de formularios propios para impedir que un sitio externo envíe peticiones en nombre del usuario. Sin él, el POST devuelve 403.
Acumulador	Variable que arranca con un valor vacío y se va llenando según el flujo. <code>respuesta</code> y <code>error</code> se pasan al template aunque no se hayan tocado.

El servicio y la IA

API	Interfaz que un programa externo expone para ser usado. El asistente consume la API de Gemini por HTTP.
Gemini	Modelo de IA de Google que responde texto. El código no lo menciona: solo conoce la URL y el modelo de settings.
generateContent	La acción concreta de la API de Gemini que genera texto. La URL termina en <code>:generateContent</code> .
Prompt	El texto que se le envía a la IA. Acá son dos parts: el mensaje de sistema y el contexto con la pregunta.
Mensaje de sistema	Instrucción de rol que define cómo debe comportarse la IA: quién es, qué responder, qué evitar. Es la guía contra las alucinaciones.
Alucinación	Cuando una IA inventa datos que no existen. Se combate dándole un contexto real y prohibiéndole responder fuera de él.
Temperature	Parámetro de generación: cuánta aleatoriedad se permite. 0.2 es bajo: respuestas predecibles y apegadas al contexto.
Timeout	Tiempo máximo de espera de una respuesta. La IA usa 15 segundos, más que el microservicio, porque generar texto es lento.
JSON	Formato de datos con llaves y listas. Es lo que el asistente envía a Gemini y lo que recibe de vuelta.

Python y Django

urllib	Librería estándar de Python para pedidos HTTP. Se usa en vez de requests para no sumar dependencias al proyecto.
Excepción propia	Clase de error definida por el proyecto, como <code>IANoDisponible</code> . Traduce fallas técnicas a un vocabulario de negocio.
Mock	Falso que reemplaza a una función real en un test. Permite probar la vista sin llamar a Gemini.
patch	Contexto de <code>unittest.mock</code> que reemplaza un objeto durante la ejecución y lo restaura al salir.
ORM	Traductor entre clases de Python y tablas SQL. <code>armar_contexto_biblioteca()</code> usa <code>Libro.objects.all()</code> para leer el catálogo real.
render()	Shortcut de Django que combina un template con un context y devuelve una respuesta HTML.
Variable de entorno	Valor externo al código (como <code>IA_API_KEY</code>). Permite cambiar la configuración sin tocar ni desplegar código.
.env	Archivo local con variables de entorno, excluido del repo. <code>settings.py</code> lo lee solo con <code>_leer_env_local()</code> .